

Covering Singapore's Bus Network via Directed Graph Traversal

Ryan Pek

April 25, 2026

1 Problem Statement

Given a directed graph $G = (V, E)$ representing bus stops and valid bus-route transitions, construct a walk starting from a chosen vertex s that visits every vertex $v \in V$ at least once.

An ideal case is to visit each vertex exactly once, which corresponds to finding a Hamiltonian Cycle or Hamiltonian Path.

However, because the network is directed and constrained by actual bus routes, my intuition is that repeated visits may be necessary to visit all the stops.

After construction of the graph, the objectives of follow up algorithms are to minimise the number of repeated vertices, such that the final walk returns a solution that approximates that of a Hamiltonian cycle.

2 Project Approach

A summary of what the project consists of:

- A data ingestion pipeline that fetches and stores raw LTA data
- A preprocessing step that constructs a compact graph representation
- A set of algorithms operating on the in-memory graph to compute covering walks

3 Data Gathering

The graph is built from LTA bus data. Raw JSON snapshots are stored first, then processed into a compact adjacency-list representation.

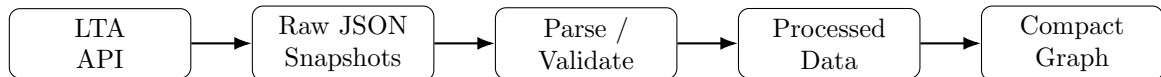


Figure 1: Data gathering and preprocessing pipeline.

4 Graph Model

Each bus stop is represented as a vertex. A directed edge (u, v) exists if a bus can travel directly from stop u to stop v along a valid service route.

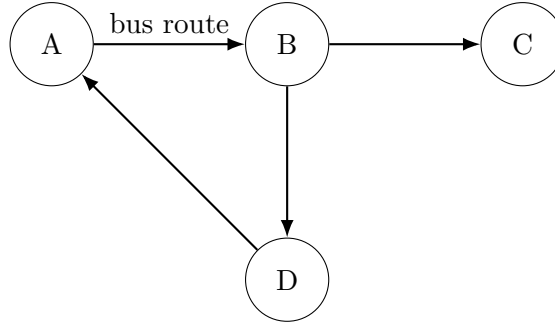


Figure 2: Directed graph model of bus-stop transitions.

5 Objectives

- Visit every bus stop at least once
- Minimise repeated visits
- Minimise total distance
- Return to the starting vertex if possible

6 Follow Up

Follow up papers will describe:

- Graph construction and validation
- Heuristic and algorithmic approaches for vertex-covering walks
- Analysis of structural properties (betweenness, flow, chokepoints)

7 Project History

The initial version of this project was implemented in Python as an exploratory prototype to understand the structure of the bus network and to experiment with basic traversal strategies.

The system consisted of multiple components:

- A set of scripts to query the LTA DataMall APIs and store responses as JSON files
- Processing functions that extracted relevant information such as bus services, routes, and stop metadata
- A large number of intermediate JSON files, including per-stop data and per-route data
- A traversal module (Traveller) that attempted to explore the graph and visit all stops

While functional, this design had several limitations:

- Heavy reliance on file-based JSON storage, resulting in thousands of small files
- Repeated parsing and scanning of JSON data during execution
- Tight coupling between data access, processing, and traversal logic
- Inefficient lookup patterns, often requiring linear scans over multiple files
- Difficulty reasoning about performance due to mixed concerns

In particular, many queries such as retrieving the direction, distance, or adjacency of a bus stop required iterating through multiple JSON datasets repeatedly. This introduced significant overhead and made the system difficult to scale.

These limitations motivated a redesign of the system, focusing on:

- Separating data ingestion, preprocessing, and algorithmic components
- Constructing a single compact in-memory graph representation
- Reducing repeated file I/O and JSON parsing
- Improving performance and clarity of the traversal algorithms

The current work represents the beginning of this redesign, with a transition to Go and a focus on building a more efficient and modular system.